

Real-Time Simulation Backend

This file explains what happens when the live stream is started, what the backend generates, and what can be inferred from the simulation.

Backend file:

- `backend/app/services/live_simulator.py`

Frontend trigger:

- `frontend/src/App.jsx`

API endpoints:

- `POST /live/start`
- `POST /live/stop`
- `POST /live/override`
- `GET /live/status`
- `GET /live/snapshot`

Why Live Simulation Exists

The project does not have direct access to:

- Real smart meter data.
- Real inverter telemetry.
- Real BMS telemetry.
- Real SCADA historian data.
- Real weather station stream.

Without live data, a dashboard would become static. To avoid that, the project includes a backend live telemetry simulator. It creates realistic, continuously changing microgrid records while preserving physical constraints.

This makes the demo behave like a control-room system instead of a static web page.

What Happens When Start Live Stream Is Clicked

Frontend action:

```
Start live stream button clicked
```

Frontend API call:

```
POST /live/start
```

```
body: { "interval_seconds": 1.5, "reset": true }
```

Backend action:

1. Creates or resets an in-memory record buffer.
2. Resets battery SoC to initial SoC, currently 55 percent.
3. Starts a background daemon thread.
4. Generates one telemetry record every 1.5 seconds.
5. Writes each record to `data/runtime/live_telemetry.jsonl`.
6. Makes the recent records available to `/forecast`, `/optimize`, `/metrics`, `/alerts`, and `/decisions`.

Frontend refresh:

```
pollMs = liveStatus.running ? 1500 : 30000
```

So when live mode is running, the dashboard refreshes every 1.5 seconds.

What Data Is Generated

Each live telemetry row contains:

Field	Meaning
<code>timestamp</code>	Local Asia/Kolkata timestamp.
<code>timestamp_utc</code>	UTC timestamp.
<code>ghi, dni, diffuse_irradiance</code>	Synthetic irradiance derived from solar output.
<code>temperature_c</code>	Simulated ambient temperature.
<code>wind_speed_mps</code>	Simulated wind speed.
<code>humidity_pct</code>	Simulated humidity.
<code>precipitation_mm</code>	Simulated precipitation.
<code>solar_kw</code>	Simulated PV power.
<code>load_kw</code>	Simulated facility demand.
<code>tariff_inr_kwh</code>	Time-of-use tariff.
<code>battery_soc_pct</code>	Current battery SoC.
<code>battery_energy_kwh</code>	Stored energy in kWh.
<code>battery_power_kw</code>	Positive means discharge, negative means charge.
<code>battery_violation</code>	0 when safe.
<code>grid_kw</code>	Required grid import.

Field	Meaning
<code>load_shed_kw</code>	Unserviced load in island mode.
<code>operator_action</code>	Auto/override dispatch action.
<code>override_mode</code>	Current operator override mode.
Time features	Hour, weekday, weekend, cyclic sine/cosine encodings.

Live Solar Logic

Solar depends on:

- Hour of day.
- Seasonal factor.
- Cloud factor.
- PV capacity.
- Performance ratio.
- Small noise.

Simplified formula:

```
daylight = max(sin((hour - 6) / 12 * pi), 0)
seasonal = 0.9 + 0.12 * sin((day_of_year - 45) / 365 * 2*pi)
solar = pv_capacity * performance_ratio * daylight^1.35 * seasonal * cloud_factor
```

Then:

```
0 <= solar_kw <= pv_capacity_kw
```

Inference:

- Solar should be near zero at night.
- Solar rises after sunrise.
- Solar falls near sunset.
- Cloud changes create realistic fluctuations.

Live Load Logic

Load depends on:

- Base load.
- Morning peak.
- Evening peak.
- Daytime activity.
- Cooling load.
- Humidity load.

- Weekend reduction.
- Occasional event spikes.
- Noise.

Simplified formula:

```
load =  
  42  
  + morning_peak  
  + evening_peak  
  + daytime_activity  
  + cooling  
  + event_spike  
  + noise
```

Then:

```
24 kW <= load_kw <= 175 kW
```

Inference:

- Morning demand rises around 6 to 10 AM.
- Evening demand rises around 6 to 10 PM.
- High temperature and humidity increase load.
- Event spikes mimic abnormal facility activity.

Live Tariff Logic

Tariff is deterministic:

```
22:00 to 05:59 -> 2.6 INR/kWh  
18:00 to 21:59 -> 9.2 INR/kWh  
all other hours -> 5.6 INR/kWh
```

Inference:

- Battery charging is cheaper at night.
- Battery discharge is most valuable in evening peak tariff.
- No random tariff jumps are used unless scenario simulation is run.

Live Battery Dispatch Logic

The simulator keeps the battery physically safe.

If override is **auto**:

```
if solar surplus and battery has room:  
    charge  
elif peak tariff/high load and battery has energy:  
    discharge  
else:  
    idle
```

If override is **force_charge**:

```
charge within max charge power and max SoC
```

If override is **force_discharge**:

```
discharge within max discharge power and min SoC
```

If override is **island**:

```
grid_kw = 0  
solar + battery serve load  
if supply is insufficient:  
    load_shed_kw > 0
```

Battery limits:

```
36 kWh <= battery_energy <= 162 kWh  
20 percent <= SoC <= 90 percent
```

What Can Be Inferred From Live Simulation

1. Whether Dispatch Is Economically Sensible

Look for:

- Charging during solar surplus or off-peak tariff.
- Discharging during peak tariff.
- Holding battery when no economic action is needed.

2. Whether Battery Safety Is Preserved

Look for:

- SoC never below 20 percent.

- SoC never above 90 percent.
- Battery violation remains 0.

3. Whether The Grid Is Being Supported

Look for:

- Lower grid import when battery discharges.
- High grid import when solar is low and battery is unavailable.
- Load shedding only in island mode when supply is insufficient.

4. Whether Alerts Make Sense

Look for:

- Peak demand alerts when load approaches high thresholds.
- Renewable drop alerts when solar falls sharply.
- Peak tariff import alerts when grid import occurs during expensive tariff.

5. Whether Operator Overrides Work

Test:

- Force charge: SoC should increase if battery has room.
- Force discharge: SoC should decrease if battery has energy.
- Island mode: grid import should become zero.
- Auto: backend resumes normal dispatch.

Difference Between Simulation And Real SCADA

Simulated project	Real deployment
Generates telemetry in backend code.	Reads telemetry from meters, inverter, BMS, and SCADA historian.
Uses synthetic cloud/load/noise models.	Uses real measured PV/load/weather data.
Writes JSONL locally.	Writes to historian, database, or message broker.
Operator override affects simulation only.	Operator override sends command to EMS/PLC/PCS through secure protocol.
Good for demo and algorithm validation.	Requires cyber-security, fail-safes, permits, and certified controls.

How To Verify It Is Running

Open:

```
http://127.0.0.1:8000/live/status
```

Expected:

```
{
  "running": true,
  "interval_seconds": 1.5,
  "records": 10,
  "override_mode": "auto"
}
```

The **records** count should keep increasing.

Runtime file:

```
data/runtime/live_telemetry.jsonl
```